Agent Build Roadmap v2 — Wine Quality MLOps (4 weeks)

Oct 13, 2026 · @kira

1. What changed from version 1, and why

We keep the same six agents, the same message board, the same decision loop and the same eight test scenarios (A to H). What changes is the project they sit on: the heavy NYC Taxi / Airflow baseline is replaced by a small wine-quality API (FastAPI + MLflow + Prometheus + Grafana), and the plan is squeezed from 10 weeks into 4.

The new baseline is BaraaZ95/FastAPI-Prometheus-Grafana-MLOPs. Its model predicts a wine's quality score (0 to 10) from 11 lab measurements. The whole dataset is about 1,600 red wines (plus about 4,900 white wines we can use later), so training takes under a second on a laptop. That is the main reason it fits in 4 weeks: nothing waits on compute, so all our time goes into the agents.

Topic	Version 1 (Taxi baseline)	Version 2 (this document)
Baseline project	Zoomcamp: Airflow, Flask, LocalStack, MLflow	Wine API: FastAPI, MLflow, MinIO, Prometheus, Grafana
Data size	~100,000 trips per daily batch	~100 wines per batch, 1,600 rows in total
Duration	10 weeks	4 weeks
Message board	Postgres, polling	One SQLite file (`board.db`), polling every 2 s
Sandbox	Separate Docker Compose project	"Dry run": a Python subprocess in a temp folder
Incident memory	Chroma vector store	One SQLite table + simple text match
Lineage	networkx graph in Postgres	One small YAML file (`dependency_map.yaml`)
Drift and label-free accuracy	Evidently + NannyML	PSI and KS test written with numpy/scipy (~40 lines)

Topic	Version 1 (Taxi baseline)	Version 2 (this document)
Serving and traffic split	Router + two serving containers	One FastAPI app holding both models, split inside the app
LLM	Local 7-8B model or API	Small local model (3B via Ollama) or an API; same switch
Experiments	4 systems × 5 repeats	2 systems required (rules vs agents), 1 optional, 3 repeats

What did not change: each agent watches one part of the pipeline, decides with an LLM (with a rule-based fallback), tries risky actions in a dry run first, asks a human for the big ones, and remembers what worked. Agents never call each other's code; they only talk through the board.

How to read this document: everyone reads sections 1 to 6 (the shared rules). Then read your own agent chapter (7 to 12) in full, and only the "needs" and "gives" tables of the other chapters. Sections 13 and 14 are for week 3 onwards.

A note on numbers: every threshold here (0.25 PSI, 10% error rise, and so on) is a starting value. Put them in `agents.yaml`, measure, and adjust.

2. The new baseline, explained

The baseline already serves and watches a model; it does not check data, retrain, review code or release safely. Those gaps are exactly where our agents go.

2.1 What is in the repository

Part	File / service	What it does today
Prediction API	`app/main.py` (FastAPI, port 8000)	`POST /predict` takes 11 numbers as a plain list and returns `{"prediction": 5.6}`. Loads one model from MLflow at startup using a hard-coded run id.

Part	File / service	What it does today
Metrics	`/metrics` on the same app	Prometheus counters and histograms: total, successful and failed requests, request latency, prediction values, CPU and memory, and input histograms for 2 of the 11 features (fixed acidity, volatile acidity).
Model registry	`mlflow` service + MySQL + MinIO	Stores runs and model files. MySQL keeps the run records, MinIO (an S3 clone, ports 9000/9001) keeps the files.
Monitoring	`prometheus` (9090), `grafana` (3000)	Prometheus scrapes the API every 5 s. Two Grafana dashboards are provisioned (FastAPI and ML metrics).
Traffic	`simulate_requests.sh`	Sends 1,000 requests with random values.
Training	none in the repo	The README trains the public `mlflow-example` (an ElasticNet regression on the red wine dataset).

The 11 input features (in this exact order): fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free sulfur dioxide, total sulfur dioxide, density, pH, sulphates, alcohol. **The target** is `quality`, an integer score given by tasters (3 to 8 in the red wine data). We treat it as regression and measure the error as MAE (mean absolute error, in quality points).

2.2 What is missing (and which agent fills the gap)

Missing	Why it matters	Agent that adds it
A training script in the repo	Nothing can retrain the model	Trainer (and the Setup Agent generates the Makefile target)
Any check on incoming data	Wrong units or missing values go straight to the model	Data Checker

Missing	Why it matters	Agent that adds it
A prediction log (each request and answer saved)	You cannot compare predictions with the real quality later	Deployer adds it to the API
Real labels after the fact	You cannot measure real error	Our replay script feeds the true `quality` with a delay
CI, schema, config for agents	No safe way to change code	Setup Agent
A second model and traffic split	Every new model goes to 100% of users at once	Deployer
Alerts that explain *why*	Grafana shows a curve, not a cause	Monitor (asks the other agents)

2.3 Small fixes we make once, in week 1

Fix	Why	Owner
Add a healthcheck to the `db` service in `docker-compose.yaml` (it is commented out, but the MLflow service waits for `service_healthy`)	Otherwise `docker compose up` can refuse to start MLflow	P1
Add `train.py` (ElasticNet in a scikit-learn Pipeline with a StandardScaler, logged to MLflow and registered as model `wine-quality`)	One reproducible training function used by every system	P4
Load the model by alias: `models:/wine-quality@champion` instead of the hard-coded run id	The Deployer can switch models without editing code	P5
Keep the `data/` folder: `winequality-red.csv` (and `winequality-white.csv`) from the UCI repository	Same data on every laptop	P3
Add `wine_id` (a row number) to every replayed request	Lets us join a prediction with its true quality later	P3 with P5

Fix	Why	Owner
Leave the input-handling bug in `predict()` as it is (see below)	It becomes scenario H, a real bug fix that changes the API on purpose	nobody

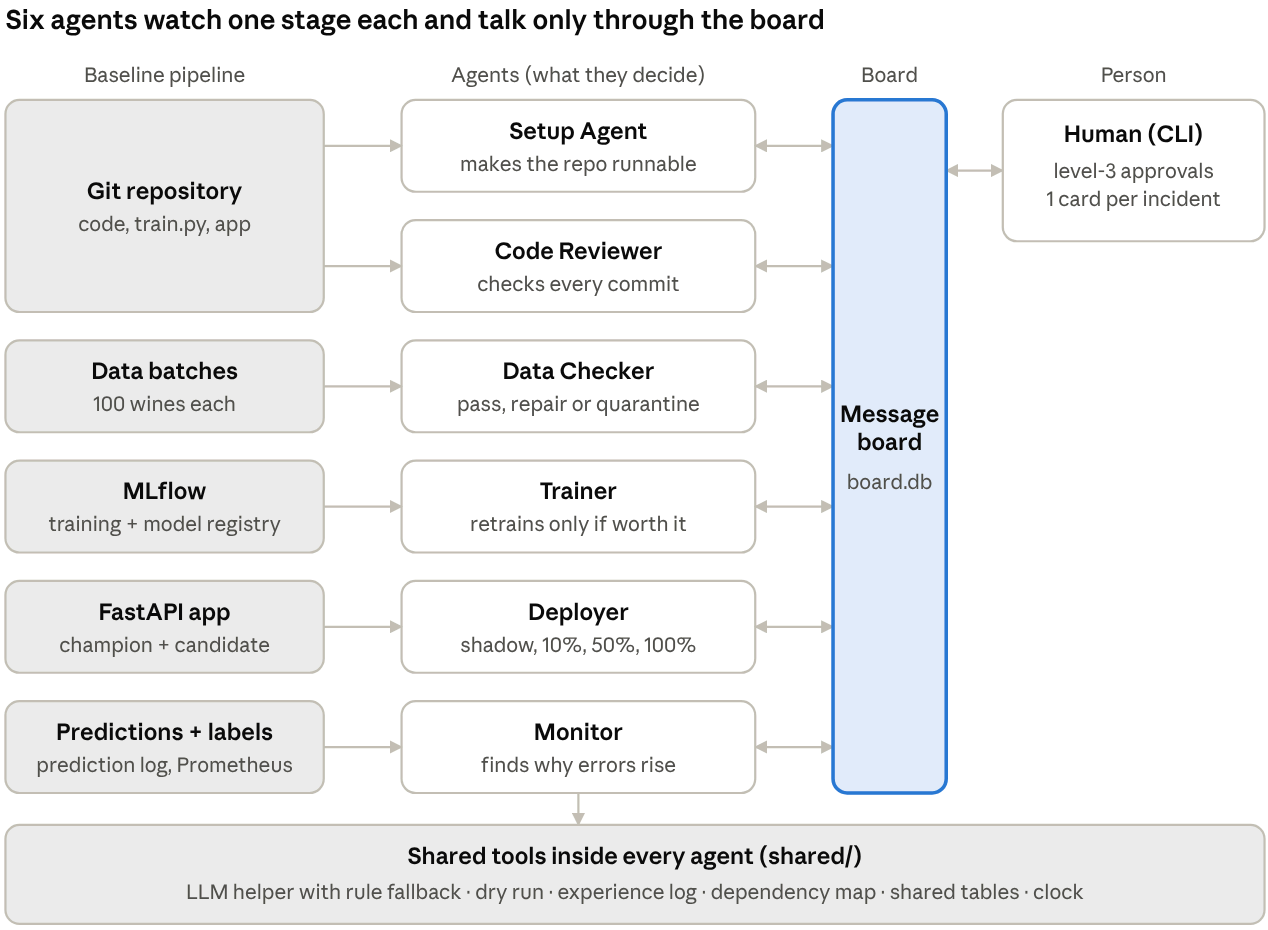
The bug we keep on purpose. In `predict()`, the `return` sits inside `finally`. If the model raises an error, `prediction_value` was never set, so the handler crashes with a confusing NameError instead of a clean 422 error. The input is also an unnamed list, so a client that sends the 11 numbers in the wrong order gets a wrong answer and no warning. Fixing both (named fields + proper error) is a deliberate API change that several agents must agree on, which is exactly what scenario H tests.

2.4 Why this baseline is light

Training ElasticNet on 1,600 rows takes well under a second. A whole "day" of replayed traffic is a few hundred requests. The full stack (API, MLflow, MySQL, MinIO, Prometheus, Grafana) fits in about 2 to 3 GB of RAM. A 3B local LLM adds about 2 to 3 GB, so the whole system runs on one 8 GB laptop. Run Ollama on the host, outside Docker, or use the API option.

3. The big picture: six agents and how they think

Six agents each watch one stage of the pipeline, decide what to do with an LLM, and coordinate only by posting messages on a shared board. No agent imports another agent's code.



system architecture · 5 pipeline stages, 6 agents, 1 board

Each agent reads its own stage on the left and talks to the others only through the board in the middle; the person is reached the same way.

3.1 The six agents

#	Agent (old name)	Watches	Main question it answers	Owner
1	Setup Agent (Builder)	The repository	"Can this repo run as a pipeline, and if the build fails, how do I fix it?"	P1
2	Code Reviewer (Reviewer)	Every commit	"Does this change break the build or silently change a feature, and was it intended?"	P2
3	Data Checker (Inspector)	Every incoming data batch	"Is this batch safe to use: pass, repair or quarantine?"	P3

#	Agent (old name)	Watches	Main question it answers	Owner
4	Trainer	Training and the model registry	"Is retraining worth it, and is the new model better for everyone?"	P4
5	Deployer	The live API and its traffic	"How much traffic does the new model get, and when do I roll back?"	P5
6	Monitor	Live predictions and their errors	"Is the model getting worse, and whose fault is it?"	P6

3.2 How every agent thinks: the 7-step loop

Every agent runs the same loop, written once in `shared/base_agent.py`. Each agent only fills in what is specific to it.

1. **Look** (`observe`): read its inputs, e.g. the new batch, the last hour of predictions, the latest commit.
2. **Understand** (`analyze`): plain statistics and checks, no LLM. Output: numbers plus a yes/no flag "something is wrong". If no flag, stop here and wait for the next round.
3. **Decide** (`decide`): the LLM receives a short summary of the numbers, similar past cases from memory, and the list of allowed actions (the *menu*). It must answer with one action from the menu, in JSON. If the LLM fails or answers nonsense twice, the agent uses its rule-based decision instead.
4. **Try safely** (`dry_run`): if the chosen action is risky (level 2, below), it is first tried in a dry run on a copy. If the dry run fails, the agent falls back to the safest action.
5. **Do** (`act`): run the action, or for level 3, ask a human and wait.
6. **Check** (`verify`): every action carries its own success check, written before acting (e.g. "the next batch passes the schema"). The agent checks it.
7. **Remember** (`remember`): save the problem, the action and whether it worked to the experience log. Next time a similar problem appears, step 3 sees it.

3.3 Action levels: how much freedom an agent has

Level	Meaning	Example	Who approves
0	Information only	Post a health report	nobody

Level	Meaning	Example	Who approves
1	Safe and easy to undo	Quarantine a batch, warn other agents, hold a release	the agent alone
2	Risky: must pass a dry run first	Repair a batch, retrain, move the new model to 25% of traffic	the dry run
3	Big or hard to undo: a person decides	Put a new model on 100% of traffic, approve a code migration	a human, through the CLI

One exception: the Deployer's **emergency rollback** is plain code, not an LLM decision, and needs no approval, because it only returns to a model a human already approved.

3.4 How agents talk

All messages go through one **message board** (a table in `board.db`). Agents post messages and read new ones every 2 seconds. There are five types:

- **finding**: "I noticed something" (e.g. `data.batch_checked`).
- **question**: "I need an answer from you" (e.g. the Monitor asks the Data Checker `query.data_changed`). Every question has a deadline; after it, the asker decides without the answer.
- **answer**: the reply to a question or request.
- **request**: "please do this" (e.g. `request.retrain`). The receiver answers accepted, refused, done or failed.
- **claim**: "this incident is mine, I found the cause" (`incident.claim`). It stops three agents from fixing the same problem.

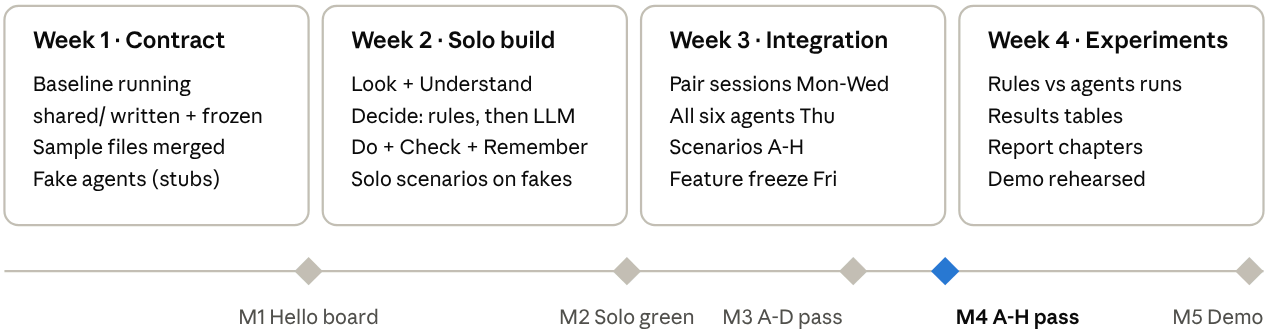
Two extra rules keep things simple. First, **messages announce, tables hold state**: anything others need later (batch status, which model is live) is also written to a shared table, so a restarted agent catches up by reading tables. Second, every problem gets one **incident id**, so all messages about the same problem are grouped, and a human gets at most one card per incident.

Typical example (scenario C). A new batch arrives with total sulfur dioxide in g/L instead of mg/L. The Data Checker sees the values are 1,000 times too small, posts `incident.claim`, quarantines the batch, tells the Trainer "do not use it" and the Deployer "hold any release". Meanwhile the Monitor notices errors going up, asks the Data Checker "did the data change?", gets "yes, unit change", and therefore does **not** request a retrain. One human card, no wasted training.

4. Team, ownership and the 4-week plan

Week 1 builds the shared contract, week 2 builds each agent alone against fake partners, week 3 plugs everything together, week 4 runs the experiments and writes the report.

Four weeks: contract, solo build, integration, experiments



4-week plan · 4 phases, 5 milestones

M4 (all eight scenarios passing with the six real agents) is the gate that matters: week 4 only measures what exists at that point.

4.1 Who owns what

Each person owns one agent and one shared piece. Heavier agents get lighter shared pieces.

Person	Agent	Shared piece they own	Load
P1	Setup Agent (hardest)	Team repo skeleton, team CI, `docker-compose.agents.yml`, compose healthcheck fix	High
P2	Code Reviewer	Dependency map (`shared/depmap.py` + `dependency_map.yaml`)	Medium
P3	Data Checker (easiest)	Base agent loop (`shared/base_agent.py`), clock, replay script and fault runner (`harness/`)	Medium-high
P4	Trainer	Dry-run runner (`shared/dry_run.py`), LLM helper (`shared/llm.py`), `train.py`	Medium-high
P5	Deployer	Experience log (`shared/memory.py`), prediction log in the API	High

Person	Agent	Shared piece they own	Load
P6	Monitor	Message board and message format (`shared/board.py`, `shared/messages.py`), human CLI, results log	Medium-high

Each person also writes the faults for their own layer (code faults: P1 and P2; data: P3; model: P4; serving: P5; monitoring: P6), in the format of section 13.4.

4.2 Week by week

Week	Goal	Every person must have by Friday	Team milestone
1: Contract	Baseline running everywhere; shared pieces written and frozen	Baseline up on your laptop (Mon-Tue). Your shared piece merged. Your sample files merged. Your agent folder with `agent.py` running on the base loop and answering a test message. Your fake agent (stub) posting its fixture messages.	**M1 "Hello board"** (Fri): six skeleton agents start from one image, receive a test message and answer it
2: Solo build	Each agent works alone against the fake versions of the other five	Mon-Tue: Look + Understand on replayed data. Wed-Thu: Decide (rules first, then LLM) + Do + Check. Fri: Remember + your solo scenarios pass.	**M2 "Solo green"** (Fri): every agent passes its scenarios against stubs
3: Integration	Replace fakes with real agents, two at a time	Pair sessions (13.2). Bugs fixed in your own agent. Thresholds moved into `agents.yaml`.	**M3** (Thu): scenarios A-D pass with all six real agents. **M4** (Fri): A-H pass; feature freeze

Week	Goal	Every person must have by Friday	Team milestone
4: Experiments and report	Measure, compare, write	Mon-Tue: experiment runs. Wed: your results table. Thu: your report chapter. Fri: demo.	**M5** (Fri): report and demo

Daily rule from week 2: merge to `main` every evening; CI must stay green. **Friday check (1 hour, weeks 2 and 3):** run `make contract-test`, run `make solo AGENT=<name>` for all six, walk through the "who needs what" table (section 6.3), and agree on any change to `shared/`.

4.3 What we cut to fit 4 weeks

These were in version 1 and are now **optional extras**, done only if a person is ahead: fine-tuning the LLM on the logged decisions, a learned risk model for code changes, a learned retrain-gain predictor, traffic forecasting and auto-scaling, GitHub-mode reviews (local git mode is enough), a web page for human decisions (the CLI is enough), and the single-agent and coordinator comparison systems.

5. The shared foundation (written together in week 1)

Everything in `shared/` is the contract: written together in week 1, frozen as v1.0 on Friday, and changed afterwards only by the process in 13.5. If every agent uses these pieces and nothing else, integration in week 3 is mostly plugging in.

5.1 Eight rules that make solo work possible

1. **Contract first.** `shared/` is written in week 1 and frozen on Friday.
2. **Agents never import each other.** An agent imports only `shared/` and its own folder.
3. **Messages announce, tables hold state.** Anything others need later is also written to a shared table.
4. **Sample files on day one.** Every file one agent makes for another gets a hand-made sample in week 1 (`samples/`). Consumers build against the sample; the producer must later produce the same shape.

5. **Every agent ships a fake (stub).** A small script that posts the agent's usual messages from fixtures and answers questions with canned replies, so the other five can test alone.
6. **Same scenarios everywhere.** Scenarios A-H are written once and run both against fakes (week 2) and for real (week 3).
7. **Contract tests in CI.** Every message is checked against the catalogue when it is posted and when it is read.
8. **One config, one image, one clock.** All agents read `agents.yaml`, run from the same Docker image (`python -m agents.<name>`), and take the time from `shared.clock.now()`, never `datetime.now()`, so replayed days behave the same for everyone.

5.2 Repository layout

```
wine-agents/
├── baseline/                # BaraaZ95 repo + our week-1 fixes (2.3)
|   ├── app/main.py          # prediction API (Deployer extends it)
|   ├── train.py             # added in week 1 (P4)
|   └── docker-compose.yaml
├── data/                    # winequality-red.csv, winequality-white.csv
├── shared/                  # THE CONTRACT
|   ├── messages.py          # message format (pydantic)
|   ├── catalogue.yaml       # every message kind: sender, receivers,
required fields
|   ├── board.py             # post / read / ask / answer, on board.db
(SQLite)
|   ├── base_agent.py        # the 7-step loop
|   ├── llm.py               # decide(): calls the LLM, checks the JSON,
falls back to rules
|   ├── dry_run.py           # dry_run(kind, inputs): try an action on a copy
|   ├── memory.py            # experience log: remember / similar /
best_fixes
|   ├── depmap.py            # dependency map: upstream / downstream / owner
|   ├── tables.py            # shared tables (batches, predictions, labels,
release_state)
|   ├── incidents.py         # open_or_join(): one incident per problem
|   ├── clock.py             # now(): replay time or real time
|   ├── config.py            # loads agents.yaml + environment variables
|   └── testing.py           # fixture loader, stub runner, scenario runner
├── agents/<name>/           # setup | reviewer | checker | trainer |
deployer | monitor
```

```
|   ├── agent.py              # class <Name>Agent(BaseAgent)
|   ├── analysis.py           # statistics and checks (no LLM here)
|   ├── actions.py            # one function per menu action
|   ├── prompt.md             # system prompt for the LLM
|   ├── stub.py + fixtures/   # the fake version of this agent
|   └── tests/
├── dry_run_jobs/<kind>.py    # one job per dry-run kind, written by the agent
that uses it
├── harness/                  # replay script, label feeder, fault runner,
experiment runner
|   └── faults/<layer>/*.yaml
├── scenarios/A.yaml … H.yaml
├── samples/                  # hand-made sample files (6.4)
├── contracts/examples/<kind>/*.json
├── agents.yaml
├── Dockerfile.agents
└── Makefile                  # contract-test, solo, scenario, experiment
```

5.3 Software

Python 3.10 (same as the baseline images). One `requirements.txt` for all agents, pinned in week 1 from a working install: pandas, numpy, scipy, scikit-learn, mlflow (same version as the MLflow server, 2.14.1, also in the API), pydantic 2, sqlalchemy 2, pyyaml, pandera, jinja2, GitPython, httpx, prometheus-client, the `ollama` client (if the local LLM is used), pytest. **Not needed any more:** Airflow, LocalStack, Postgres, Chroma, networkx, Evidently, NannyML, LightGBM, SHAP.

5.4 Run modes

Mode	Board	Other agents	LLM	Dry run	Command
unit	in memory	none	fake (scripted answers)	in-process	`pytest agents/<name>`
solo	`board.db`	fakes of the other five	real or fake	subprocess	`make solo AGENT=checker SCENARIO=C`
integration	`board.db`	all real	real	subprocess	`make scenario S=C`
experiment	fresh `board.db` per run	per system config	real	subprocess	`make experiment SYSTEM=agents REPEATS=3`

5.5 The message format

One envelope for every message. Keep `summary` for humans and `payload` for machines.

Field	Type	Required	Meaning
`id`	text	yes	`msg-` + 12 hex characters, made by the sender
`time`	ISO-8601 UTC	yes	from `clock.now()`
`type`	finding / question / answer / request / claim	yes	see 3.4
`kind`	text	yes	exact name from the catalogue, e.g. `data.batch_checked`. An answer repeats the kind it answers
`sender`	agent name, `human` or `harness`	yes	who posted it
`to`	list	yes	named receivers; empty = everyone subscribed to the topic
`topic`	text	yes	what it is about, as a dependency-map node (5.6), e.g. `feature:total_sulfur_dioxide`
`incident_id`	text or null	no	`inc-` + 8 hex, only from `incidents.open_or_join()`
`reply_to`	text or null	answers	id of the question or request answered
`level`	0-3	yes	level of the action taken or requested
`confidence`	0.0-1.0 or null	no	how sure the sender is
`summary`	text, max 160 characters	yes	one sentence a human can read
`evidence`	object	yes	the numbers behind the message (can be `{}`)

Field	Type	Required	Meaning
`payload`	object	yes	fields the receiver acts on, checked against the catalogue
`status`	open / answered / done / expired	yes	changed only by the board
`expires_at`	ISO-8601 or null	questions, requests	after this, the sender stops waiting
`run_id`	text	yes	experiment run id (`dev` outside experiments)

Example, the Monitor asks the Data Checker (scenario C):

```
{
  "id": "msg-3f9a1c2b7d04", "time": "2026-03-14T10:42:11Z",
  "type": "question", "kind": "query.data_changed",
  "sender": "monitor", "to": ["checker"],
  "topic": "feature:total_sulfur_dioxide", "incident_id": "inc-5b21e0aa",
  "reply_to": null, "level": 1, "confidence": 0.78,
  "summary": "Live MAE up 22% since 06:00; drift concentrated on
total_sulfur_dioxide",
  "evidence": {"mae_now": 0.61, "mae_ref": 0.50, "psi":
{"total_sulfur_dioxide": 0.93}},
  "payload": {"feature": "total_sulfur_dioxide", "start": "2026-03-
14T06:00:00Z", "end": "2026-03-14T10:40:00Z"},
  "status": "open", "expires_at": "2026-03-14T11:42:11Z", "run_id": "exp-03-
agents-r2"
}
```

The answer has `type: answer`, the same `kind`, `reply_to: "msg-3f9a1c2b7d04"` and a payload like `{"changed": true, "batch_ids": ["batch-0042"], "likely_cause": "unit_change"}`.

Rules of use: put every number you reasoned on in `evidence`. Never put raw data rows in a message: save them to a table or file and put a reference (`batch_id`, `mlflow_run_id`, path) in the payload. Never put the injected fault's answer in a message; only the harness knows it.

5.6 The dependency map (owner P2)

A small YAML file that says what depends on what, so the Monitor can walk upstream from a bad feature to the code, data and model behind it. Its node names are also the message topics.

Node	Format	Example
Code file	`code:<path>`	`code:baseline/app/main.py`
Feature	`feature:<name>`	`feature:alcohol`
Data batch	`dataset:<batch_id>`	`dataset:batch-0042`
Model version	`model:<name>@<version>`	`model:wine-quality@3`
Deployment	`deployment:prod`	`deployment:prod`
Other topics (not in the map)	`commit:<sha7>`, `pipeline:setup`, `incident:<id>`, `expected_change:<id>`	`commit:abc1234`

Links: code → feature ("computes"), feature → dataset ("column of"), dataset → model ("trained on"), model → deployment ("serves"). API: `depmap.upstream(node)`, `depmap.downstream(node)`, `depmap.owner(node)`, `depmap.add_link(src, dst, relation)`.

5.7 Board API (owner P6)

Call	What it does
`board.post(msg) -> id`	Checks the message against the catalogue and saves it. An invalid message raises an error: fix it, don't hide it.
`board.read(agent) -> list[Message]`	New messages for this agent: named in `to`, or `to` empty and the topic matches its subscriptions. Remembers where the agent stopped reading (in the `cursors` table).

Call	What it does
`board.ask(msg, timeout_s) -> Message or None`	Posts a question or request and waits for the answer until `expires_at`. `None` means no answer; note that in `evidence` and decide anyway.
`board.answer(original, **fields)`	Builds the answer (same kind, `reply_to`, `to=[original.sender]`) and marks the original answered.
`board.thread(incident_id)`	All messages of one incident, in order (for human cards and learning).
`board.heartbeat(agent)`	"I'm alive"; the base loop calls it every round.
`incidents.open_or_join(nodes, window)`	Joins an open incident on the same branch of the dependency map within the time window; otherwise opens a new one.

Handlers must be **idempotent**: keep the ids of messages already handled and skip repeats, because a crash can deliver the same message twice.

5.8 Base agent (owner P3)

```
class BaseAgent:
    name: str
    menu: list[Action]          # Action(name, level, params_schema,
success_check)

    # ---- each agent writes these ---------------------------------
    def observe(self) -> dict: ...                # 1. Look
    def analyze(self, obs) -> Analysis: ...       # 2. Understand: numbers +
flag + nodes
    def context(self, analysis, inbox) -> dict:   # what the LLM sees (no raw
rows)
    def rule_decision(self, analysis) -> Decision # fallback AND the 'rules
only' system
    def act(self, decision) -> Result: ...        # 5. Do: calls actions.py
    def handle(self, msg) -> None: ...            # questions and requests
from others
```

```
    # ---- given by the base class ---------------------------------
    def step(self):
        for msg in board.read(self.name): self.safe_handle(msg)
        analysis = self.analyze(self.observe())
        if not analysis.flag: return board.heartbeat(self.name)
        incident = incidents.open_or_join(analysis.nodes, cfg.window)
        decision = llm.decide(self, self.context(analysis, self.inbox),
                              self.menu,
fallback=self.rule_decision(analysis))
        if decision.level == 2 and not dry_run(decision.dry_run_kind,
decision.params).ok:
            decision = self.rule_decision(analysis).safest()
        if decision.level == 3:
            return self.ask_human(incident, decision)
        result = self.act(decision)
        ok = self.verify(decision, result)
        memory.remember(self.name, incident, analysis, decision, result, ok)
        log.action(self.name, incident, decision, result, ok)
```

`rule_decision()` does double duty: it is the safe fallback when the LLM fails, and running all agents with `reason: rules` and the board off **is** the traditional pipeline we compare against, at almost no extra cost.

5.9 LLM helper (owner P4)

`llm.decide(agent, context, menu, fallback)` is the only way an agent calls an LLM. The LLM must answer:

```
{ "action": "quarantine",
  "level": 1,
  "params": {"batch_id": "batch-0042"},
  "reason": "total_sulfur_dioxide median is 1/1000 of reference; matches past
case 'g/L instead of mg/L'",
  "confidence": 0.86,
  "success_check": "next batch passes schema and PSI < 0.25 on
total_sulfur_dioxide",
  "memory_refs": ["mem-0012"] }
```

- Temperature 0, JSON output enforced (Ollama `format: json` or the API's structured output).
- Checked with pydantic: unknown action, wrong level or bad params → one retry with the error added; a second failure → the rule decision, logged as `llm_invalid`.

- The context holds numbers and memory matches only, never raw rows, and is capped at about 2,000 tokens.
- `llm.provider: ollama | api` and `llm.model` in `agents.yaml`, so local vs API is a config switch.
- Every call is logged in `llm_calls` (agent, incident, prompt, raw answer, parsed decision, tokens, latency). This is also the dataset if someone tries fine-tuning later.

5.10 Dry run, experience log and clock

Dry run (owner P4). `dry_run(kind, inputs, timeout_s=120) -> DryRunResult(ok, outputs, metrics, log_tail, seconds)`. It runs `dry_run_jobs/<kind>.py` in a subprocess, in a temp folder, with `DRY_RUN=1` set. `shared/tables.py` refuses to write to real tables when `DRY_RUN=1`, and MLflow runs go to an experiment called `dry_run`. It gives isolation, not logic: each agent writes its own job.

Kind	Written by	Inputs → outputs
`build`	Setup Agent	repo path, generated files → stage reached, error class, log tail
`feature_diff`	Code Reviewer	old and new code, sample rows → per-feature difference report
`repair_check`	Data Checker	batch, repair + params → repaired copy, checks passed, PSI after
`train_compare`	Trainer	data window → candidate run id, overall + per-group comparison
`traffic_replay`	Deployer	model, recorded requests, speed → latency, errors
`confirm_replay`	Monitor	time window, model version → MAE on labelled recent data

Experience log (owner P5). One SQLite table `memory` plus three functions: `memory.remember(agent, incident_id, problem, evidence, action, params, level, outcome, time_to_recover_s) -> mem_id`, `memory.similar(problem, agent=None, k=5)` (match on error class + feature + word overlap; no vector database needed), and `memory.best_fixes(problem, agent) -> [(action, params, success_rate, n)]`.

Clock (owner P3). `clock.now()` returns the replay time during experiments (1 real second = 12 simulated minutes, set in `agents.yaml`) and real UTC time otherwise.

5.11 Shared tables: who writes, who reads

Table	Written by	Read by	Main columns
`batches`	replay script (new rows `pending`), Data Checker (status)	Trainer, Monitor	batch_id, path, status (pending / passed / repaired / quarantined), checked_at, reason
`batch_stats`	Data Checker	Data Checker, Monitor	batch_id, column, null_rate, mean, median, p1, p99
`predictions`	the API (Deployer)	Monitor, Deployer	request_id, wine_id, time, model_version, variant (champion / candidate / shadow), features (JSON), prediction, latency_ms, status_code
`labels`	label feeder	Monitor, Deployer, Trainer	wine_id, quality, available_at (delayed by `labels.delay`)
`release_state`	Deployer	the API, Monitor, Trainer	champion_version, candidate_version, stage, candidate_share, shadow_on, updated_at
MLflow model `wine-quality`	Trainer (versions, alias `candidate`), Deployer (alias `champion`)	Deployer, Monitor, Data Checker	model + `reference/` files (6.4)
`memory`	every agent through the base loop	every agent	5.10
`runs`, `llm_calls`, `actions`	base loop, harness	analysis notebooks	the experiment log

Table	Written by	Read by	Main columns
`faults`	harness only	evaluation scripts only	ground truth of injected faults

All tables live in `board.db` except MLflow. One file means every laptop runs the same thing with no database server.

5.12 Human cards (owner P6)

`human` is a receiver like any agent. A `request.approval` message produces one card per incident: what happened, the cause, the owner agent, actions already taken, the decision needed with options and a default. A small CLI answers it: `python -m shared.human list | show inc-… | approve inc-… | reject inc-…`.

5.13 agents.yaml (sample version)

```
run_id: dev
db_url: sqlite:///board.db
clock: {mode: replay, speed: 720}          # 1 real second = 12 simulated
minutes
llm: {provider: ollama, model: "<chosen in week 1>", temperature: 0,
      base_url: "http://host.docker.internal:11434"}
mlflow: {tracking_uri: http://localhost:5000, model_name: wine-quality}
api: {url: http://localhost:8000, metrics: http://localhost:8000/metrics}
prometheus: {url: http://localhost:9090}
labels: {delay: 1d}
batch_size: 100
features: [fixed_acidity, volatile_acidity, citric_acid, residual_sugar,
chlorides,
           free_sulfur_dioxide, total_sulfur_dioxide, density, pH, sulphates,
alcohol]
units: {chlorides: g/dm3, free_sulfur_dioxide: mg/dm3, total_sulfur_dioxide:
mg/dm3,
        density: g/cm3, sulphates: g/dm3, alcohol: "% vol"}
groups: [alcohol_band, acidity_band, quality_band]
agents:
  checker:  {subscriptions: ["dataset:*", "feature:*", "expected_change:*"],
             thresholds: {psi_warn: 0.10, psi_alert: 0.25}}
  trainer:  {subscriptions: ["dataset:*", "model:*", "expected_change:*"],
             thresholds: {min_gain_pct: 2, max_group_worse_pct: 5,
retrain_every_batches: 10}}
```

```
  deployer: {subscriptions: ["model:*", "deployment:*", "expected_change:*"],
             thresholds: {p95_ms: 300, error_rate: 0.05}}
  monitor:  {subscriptions: ["feature:*", "model:*", "deployment:*",
"expected_change:*"],
             thresholds: {mae_rise_pct: 10, windows: 3, weighted_psi: 0.2}}
  reviewer: {subscriptions: ["code:*", "feature:*"]}
  setup:    {subscriptions: ["pipeline:*"]}
```

6. Message list and dependencies

An agent may post only the kinds listed for it, and must handle every kind addressed to it. `catalogue.yaml` holds the same list with full field types; `contracts/examples/<kind>/` holds at least one example per kind, written by the sender and checked by each receiver.

6.1 Message catalogue (v1.0)

Kind	Sender	Receivers	Topic	Required payload fields
Findings				
`pipeline.setup_completed`	setup	all	`pipeline:setup`	repo, commit, files{}, agents_yaml, schema_path, depmap_seeded
`pipeline.setup_failed`	setup	human	`pipeline:setup`	stage, error_class, attempts, log_ref
`code.change_assessed`	reviewer	subscribers	`commit:<sha>`	commit, risk_score, verdict, affected_features[], one_sided, feature_diff{}
`data.batch_checked`	checker	trainer, monitor	`dataset:<id>`	batch_id, status, rows, failed_checks[], psi{}, repair
`data.drift_trend`	checker	monitor, trainer	`feature:<name>`	feature, statistic, slope_per_batch, batches_to_limit
`monitor.health_report`	monitor	subscribers	`model:<n>@<v>`	window, mae_now, mae_ref, weighted_psi, by_group{}
`monitor.degradation_detected`	monitor	checker, trainer, deployer	`model:<n>@<v>`	metric, reference_value, current_value, drop_pct, source (labels / drift), window, top_features[], groups[]
`training.decision`	trainer	monitor	`model:wine-quality`	decision (retrain / refuse), expected_gain, threshold, data_window, reason_code
`model.candidate_rejected`	trainer	monitor, human	`model:<n>@<v>`	version, overall_delta, failing_groups[]

Kind	Sender	Receivers	Topic	Required payload fields
`deployment.stage_changed`	deployer	monitor, trainer	`deployment:prod`	model_version, from_stage, to_stage, candidate_share
`deployment.rolled_back`	deployer	all	`deployment:prod`	from_version, to_version, trigger{metric, value, limit}, automatic
`incident.closed`	incident owner	all	`incident:<id>`	outcome, root_cause_node, actions[], time_to_recover_s
Requests				
`request.retrain`	monitor	trainer	`model:wine-quality`	reason, window, affected_features[], expected_change_id
`request.hold`	monitor or incident owner	deployer	`deployment:prod`	reason, release_condition
`request.do_not_use`	checker	trainer	`dataset:<id>`	batch_ids[], reason
`request.clean_window`	trainer	checker	`dataset:window`	start, end, min_rows
`request.deploy_candidate`	trainer	deployer	`model:<n>@<v>`	model_uri, version, champion_version, metrics{overall, by_group}, comparison_ref
`request.rebuild`	reviewer	setup	`pipeline:setup`	reason, changed_paths[]
`request.expected_change`	reviewer	checker, monitor, trainer, deployer	`expected_change:<id>`	change_id, commit, changes[{name, old, new, factor}], tasks{agent: task}
`request.approval`	incident owner	human	`incident:<id>`	question, options[], default_if_no_answer, card_ref
Questions				
`query.data_changed`	monitor	checker	`feature:<name>`	feature, start, end
`query.code_changed`	monitor, checker	reviewer	`feature:<name>`	feature, since
`query.deployment_changed`	monitor	deployer	`deployment:prod`	since
Claims and decisions				
`incident.claim`	any agent	all	`incident:<id>`	root_cause_node, evidence_refs[], instructions{agent: action}
`human.decision`	human	requester	`incident:<id>`	decision (approve / reject), comment

6.2 Answer payloads

Answering	Answer payload
`query.data_changed`	changed (bool), batch_ids[], likely_cause (unit_change / schema_change / missing_values / late_batch / valid_shift / none)
`query.code_changed`	commits[{sha, time, verdict, feature_changed, intended}]
`query.deployment_changed`	changes[{time, from_stage, to_stage, model_version}]
any `request.*`	decision (accepted / refused / done / failed), detail, plus: `request.clean_window` → batch_ids[], rows · `request.deploy_candidate` → rollout_id · `request.expected_change` → task, done_at

6.3 Who needs what from whom

This table is the agenda of the Friday check. Every dependency has a stand-in, so nobody waits.

Who needs it	What	From	Stand-in until then	Real by
all	Board, message format, catalogue	P6	in-memory board	W1
all	Base loop, clock	P3	none	W1
all	LLM helper	P4	fake LLM with scripted answers	W1
all	`agents.yaml`	Setup Agent	`samples/agents.yaml`	W1 sample, W3 generated
Reviewer, Monitor	Dependency map	P2	`samples/dependency_map.yaml`	W1

Who needs it	What	From	Stand-in until then	Real by
all	Experience log	P5	in-memory list	W1
Data Checker	Pandera schema for a batch	Setup Agent	`samples/schema.py`	W1 sample, W2 generated
Data Checker, Monitor	Reference data and PSI bins of the live model	Trainer	`samples/reference/`	W2
Monitor, Deployer	Prediction log	Deployer	`samples/predictions.parquet`	W2
Monitor, Trainer	Delayed labels	P3 (label feeder)	`samples/labels.parquet`	W2
Trainer, Monitor	Batch status	Data Checker	sample registry, all passed	W2
all	Dry-run runner	P4	in-process call	W1
all	Replay script, fault runner	P3	fixtures only	W2
Deployer	Candidate model	Trainer	a hand-registered version	W2
Trainer	Retrain requests	Monitor	Monitor stub	W2
Monitor	Answers about data, code, releases	Checker, Reviewer, Deployer	stubs with canned answers	W2
Checker, Monitor, Trainer, Deployer	Expected-change plan	Reviewer	Reviewer stub	W2
Deployer	Hold requests	Monitor, Checker	stubs	W2

Who needs it	What	From	Stand-in until then	Real by
Deployer, owners	Human decisions	P6 (CLI)	auto-approve in stub mode	W2
Monitor, Trainer	Release events	Deployer	Deployer stub	W3
Reviewer	Incident outcomes	owners	fixtures	W3
Setup Agent	Rebuild requests	Reviewer	fixture	W3

6.4 Sample files (made by hand in week 1)

File	Made by	Checked by	Content
`samples/agents.yaml`	P1	all	section 5.13
`samples/schema.py`	P1 + P3	P3	Pandera schema `WineBatch` (11 columns, ranges, no nulls)
`samples/dependency_map.yaml`	P2 + P1	P6	code → feature → dataset links for the baseline
`samples/reference/`	P4	P3, P6	from one training run: `reference.parquet` (the 400 holdout rows with prediction and label), `feature_importance.json` (absolute scaled ElasticNet coefficients), `psi_bins.json`, `group_metrics.json`
`samples/predictions.parquet`	P5	P6	one simulated day of API predictions

File	Made by	Checked by	Content
`samples/labels.parquet`	P3	P6	the true quality for those wines with `available_at`
`samples/feature_sample.parquet`	P2	P2	500 clean rows used by every feature-diff test
`contracts/examples/<kind>/*.json`	senders	receivers	at least one valid example per kind

7. Agent 1: Setup Agent (Builder) · owner P1

The Setup Agent turns an ML repository into a pipeline the other agents can plug into, and fixes its own build errors (at most 3 attempts). Difficulty: hardest. Runs once at setup, then on `request.rebuild`.

7.1 What it must do

MVP: for the wine baseline, generate the missing pieces: `Makefile` (targets `install`, `train`, `serve`, `test`, `all`), a CI workflow (`.github/workflows/pipeline.yml`: lint with ruff + pytest), `agents.yaml`, `schema.py` and the dependency-map seed. Build the result in a dry run, read the error, apply a fix, retry. Also run it on **two** tiny scikit-learn repos you write yourself (one with `train.py` / `predict.py` in the root, one with a `src/` folder and no requirements file).

Extras: more test repos, notebook-only repos, `pyproject.toml` projects. **Out of scope:** Kubernetes, GPU images, non-Python projects.

7.2 Needs and gives

Needs	From	Stand-in
Dry-run job `build`	P4 runner, own job	run `make all` in a temp folder
Dependency map API	P2	write the YAML directly
Experience log	P5	in-memory list
`request.rebuild`	Reviewer	fixture

Gives	To
`agents.yaml` (paths, endpoints, features, units, subscriptions)	all
`schema.py` with the Pandera schema `WineBatch`	Data Checker
Dependency-map seed (code → feature → dataset)	Reviewer, Monitor
CI workflow	Reviewer (CI results are its input)
Makefile targets with fixed names	Trainer, dry-run jobs
`pipeline.setup_completed` / `pipeline.setup_failed`	all / human

7.3 Action menu

Action	Level	Success check
`write_files`	1	every generated file parses; `docker compose config` passes
`dry_run_build`	1	`make all` exits 0 on a 200-row sample
`apply_fix` (max 3 times)	2	the stage that failed now passes in the dry run
`commit_setup`	1	commit exists (after go-live: a branch the Reviewer reviews)
`give_up_and_report`	1	`pipeline.setup_failed` posted with stage, error class and log

7.4 Build steps

When	Step	Done when
W1 Mon-Tue	Run the baseline by hand. Write down every manual step (compose fix, MinIO bucket, train, copy run id, restart app). That list is your requirements.	List in your README
W1 Wed-Fri	Sample files with their users: `agents.yaml`, `schema.py` (with P3), dependency-map seed (with P2), CI workflow. Team repo skeleton and team CI.	Every other agent runs on the samples
W2 Mon	**Repo scanner** → `RepoProfile` JSON using Python's `ast`: imports, entry points (`__main__`, files named train/predict/app, FastAPI app objects), data paths (strings ending `.csv`), model outputs (`mlflow.*.log_model`, `joblib.dump`), the feature list (column names in `pd.DataFrame(columns=[...])`).	Profile of the baseline finds the API, the 11 features and the MLflow model

When	Step	Done when
W2 Tue	**Dependencies:** use `requirements.txt` if present, else imports minus the standard library (`sys.stdlib_module_names`), mapped with a small table (`sklearn` → scikit-learn, `yaml` → PyYAML).	Both test repos get an installable list
W2 Wed	**Templates** (Jinja) for Makefile, Dockerfile, CI, `agents.yaml`. Fill from the profile. Ask the LLM only for blanks the scanner could not fill, with the allowed values listed. **Schema:** `pandera.infer_schema` on the CSV, then widen ranges to the 0.1-99.9 percentiles ± 10%.	Templates render for all 3 repos; schema accepts normal batches and rejects the data faults
W2 Thu-Fri	**Build-and-fix loop:** classify the last 50 log lines with regex first (`ModuleNotFoundError`, `ResolutionImpossible`, `FileNotFoundError`, exit 137 = out of memory), look up the experience log, then let the LLM pick a fix. Fix menu: `add_dependency`, `pin_version`, `change_base_image`, `add_system_package`, `fix_path`, `change_entry_command`.	3 repos build in ⩽ 3 attempts
W3	Generate for the baseline, compare with the samples, push CI, post `pipeline.setup_completed`. Handle `request.rebuild` by opening a branch instead of committing.	Generated `agents.yaml` matches the sample on required keys; CI green
W4	Measure on the 3 repos: success rate, attempts, minutes; once with the experience log and once without.	Results table

7.5 What the LLM sees

Repo profile summary, the failing stage, the error class, the last 30 log lines, similar past fixes with success rates, the fix menu. Answer example: `{"action": "apply_fix", "params": {"fix": "add_dependency", "package": "psutil"}}`.

7.6 Fake version, faults, tests, risks

- **Fake:** posts `pipeline.setup_completed` pointing at the sample files.
- **Faults you add:** broken import, missing dependency, wrong data path, unpinned package.
- **Tests:** baseline files pass `docker compose config`; CI green; `agents.yaml` matches the sample; 3 repos build in ⩽ 3 attempts; every attempt is in the experience log.
- **Risks:** open scope (treat unknown layouts as `give_up_and_report`, which is a valid result); slow builds (pip cache, slim image, 200-row sample); others wait on your samples in week 1, so do those first.

8. Agent 2: Code Reviewer (Reviewer) · owner P2

The Code Reviewer checks every commit: does it break the build, does it silently change how a feature is computed, and was that change intended? If it was intended, it coordinates the other agents through a migration plan. Difficulty: medium. Runs on every commit.

8.1 Why this agent matters here

The feature logic exists twice: once in `train.py` (reads the CSV, columns like `"total sulfur dioxide"`) and once in `app/main.py` (turns the 11-number list into a DataFrame). If someone changes one side only (a unit, a column order, a rename), the model gets different inputs in training and in production. That is **training–serving skew**, and nothing in the baseline would notice.

Week-1 fix (P2 with P4 and P5): move both into small pure functions, `prepare_features_train(df)` in `train.py` and `prepare_features_serve(values)` in `app/main.py`, so the Reviewer can call both on the same sample rows. Keep them separate: that duplication is exactly the risk we want to catch.

MVP: rule risk score; feature-diff test (old vs new, and train vs serve); LLM judgement of intent; verdict posted on the board and written to a status file; migration plan via `request.expected_change`; answers to `query.code_changed`. Local git mode only.

Extras: GitHub PR comments, a learned risk model.

8.2 Needs and gives

Needs	From	Stand-in
feature list and units	Setup Agent (`agents.yaml`)	sample
CI result per commit	Setup Agent's workflow	local `make test`
Dry-run runner	P4	in-process
Answers to the migration plan	Checker, Monitor, Trainer, Deployer	stubs answer `done`
`incident.closed` with a `code:` cause	incident owners	fixtures

Gives	To
`code.change_assessed`	Checker, Monitor, Trainer
`request.expected_change` (migration plan)	Checker, Monitor, Trainer, Deployer
answers to `query.code_changed`	Monitor, Checker
`request.rebuild`	Setup Agent
dependency map (shared piece)	everyone

8.3 Action menu

Action	Level	Success check
`approve`	1	CI green after merge and no feature changed outside a plan
`run_extra_tests`	1	selected tests ran, result attached
`ask_author`	1	question posted; status `pending` until the author answers
`block`	1	status `failure` with the reason

Action	Level	Success check
`approve_with_migration`	3	a human confirms the plan; all four agents answer `done`
`warn_agents`	1	`code.change_assessed` delivered
`request_rebuild`	1	Setup Agent answers `accepted`

8.4 Build steps

When	Step	Done when
W1 Mon-Tue	Reproduce the `predict()` bug (send a request that makes the model fail and watch the NameError). Read both feature paths line by line.	Short note with the bug
W1 Wed-Fri	Dependency map (shared): YAML file + `upstream`, `downstream`, `owner`, `add_link`. Extract the two feature functions. Make `samples/feature_sample.parquet`.	Monitor stub can walk upstream from `feature:alcohol`
W2 Mon	**Change intake:** poll local branches with `git fetch`; parse each diff into files and touched functions (match line ranges to `ast` function ranges).	Every new commit gives a `ChangeSet`
W2 Tue	**Risk score** 0 to 1: feature function touched +0.35; only one of train/serve touched +0.25; requirements or Dockerfile +0.15; `agents.yaml` or config +0.10; diff > 200 lines +0.05; file with past incidents +0.10.	Scores for 15 sample commits
W2 Wed	**`feature_diff` dry-run job:** run the old and new versions of both functions on the 500 sample rows. Per column: dtype, share of identical values, median ratio new/old. Also train vs serve on the same rows. Load each function without importing its module: `git show <sha>:<path>`, take the function's `ast` node, compile it alone.	mg → g change shows ratio ≈ 0.001; a one-sided change is flagged as skew

When	Step	Done when
W2 Thu	**Intent signals:** commit message tag `intent: total_sulfur_dioxide mg -> g`; conventional prefix (`fix:` , `feat:`); schema changed in the same commit; both sides changed the same way.	Signals extracted for test commits
W2 Fri	**LLM decision** using the situation table: intended + complete → migration; intended + incomplete → ask with the missing list; no intent → ask; clear bug → block. Migration plan: one task per agent; track the four answers; on timeout, open a human card.	Scenarios A, B, H pass solo
W3	Answer `query.code_changed` . Count incidents per file from `incident.closed` . Run 10 harmless commits (docs, renames) to measure false blocks.	No false block on the 10 harmless commits

8.5 What the LLM sees

Risk factors with weights, the feature-diff report (changed columns only), commit message, intent signals found, whether the schema changed, whether both sides changed the same way, incident history of the files, similar past reviews. Output: one menu action, plus for `ask_author` the question in one sentence quoting the measured ratio.

8.6 Fake version, faults, tests, risks

- **Fake:** posts `code.change_assessed` (approve) and a `request.expected_change` for `total_sulfur_dioxide` mg → g; answers `query.code_changed` with "no change" or a canned commit.
- **Faults you add** (git patches applied on a branch): accidental unit change on the serving side, intended unit change (both sides + schema + tag), one-sided column reorder, dependency bump, broken import, the `predict()` bug fix.
- **Tests:** A is blocked or asked within one review with the ratio reported; B completes with four `done` answers; one-sided change blocked; H approved with migration; no false block on harmless commits.
- **Risks:** importing `app/main.py` starts MLflow loading, so never import it (use the `ast` trick).

9. Agent 3: Data Checker (Inspector) · owner P3

The Data Checker decides for every incoming batch: pass, pass with a warning, repair, or quarantine, so bad data never reaches training or the live model. Difficulty: easiest. Runs on every batch.

9.1 How data flows in our replay

The replay script shuffles the wine CSV and cuts it into **batches of 100 wines** (`batch-0001` , `batch-0002` , …), one per simulated hour. For each batch it (1) saves the file and adds a `pending` row to `batches` , (2) sends the same 100 wines to the API as live requests, and (3) after `labels.delay` writes their true quality to `labels` . A data fault is applied to the feed, so it reaches both the batch and the live requests, just like in real life.

Severity depends on the model. Weight every alert by the feature's importance (from `feature_importance.json`). A big shift on a feature the model barely uses should give a warning, not a quarantine. That is a good test of judgement (scenario E).

MVP: schema checks (Pandera), batch statistics, PSI and KS drift, Isolation Forest on batch statistics, four repairs tested in a dry run, LLM decision, batch registry, answers to `query.data_changed` , incident claims. **Extras:** drift trend prediction, schema update after an approved migration.

9.2 Needs and gives

Needs	From	Stand-in
Pandera schema `WineBatch`	Setup Agent	sample
Reference data, importance, PSI bins	Trainer	`samples/reference/`
new batches as `pending` rows	replay script (own shared piece)	fixtures
`query.data_changed`	Monitor	Monitor stub
`request.clean_window`	Trainer	Trainer stub
`request.expected_change`	Reviewer	Reviewer stub

Gives	To
batch status in `batches` (Trainer only reads passed and repaired)	Trainer, Monitor
`data.batch_checked`, `data.drift_trend`	Trainer, Monitor
`request.do_not_use`, `request.hold` (as incident owner)	Trainer, Deployer
`incident.claim` for data-caused incidents	all
answers to `query.data_changed`, `request.clean_window`, `request.expected_change`	Monitor, Trainer, Reviewer
base loop, clock, replay script, fault runner (shared)	everyone

9.3 Action menu

Action	Level	Success check
`pass`	1	registry row `passed`
`pass_with_warning`	1	row `passed`, warning posted
`quarantine`	1	batch absent from the next training window
`repair`	2	repaired copy passes all checks and PSI < 0.25 on the repaired feature
`warn`	1	`data.drift_trend` or finding delivered
`request_hold`	1	Deployer answers `accepted` (incident owner only)
`update_schema`	2	only after `request.expected_change`; next batch passes

9.4 Build steps

When	Step	Done when
W1 Mon-Tue	Profile the red wine CSV: ranges, medians, correlations. Write `samples/schema.py` with P1 and `samples/labels.parquet`.	Profile notebook
W1 Wed-Fri	**Base loop (shared)** with P6 and P4: 7-step loop, menu, levels, dry-run gate, logging, memory hook, heartbeat, clock.	M1: six skeleton agents run on it
W2 Mon	**Replay script (shared):** batches, live requests with `wine_id`, delayed labels, clock that advances. **Schema checks** (Pandera, lazy mode): 11 columns and types, ranges, no nulls, pH 2.5-4.5, density 0.98-1.01, new or missing columns flagged. **Batch statistics** to `batch_stats`.	One simulated day replays end to end
W2 Tue	**Drift:** PSI per feature against the reference bins (10 bins), KS test for each numeric column, 0.10 warn / 0.25 alert. **Isolation Forest** (scikit-learn) on the statistics vectors of the first 10 normal batches.	⩽ 1 false alarm on 15 normal batches
W2 Wed	**Repairs** as small pure functions: `rescale(column, factor)` when the median ratio is close to 10, 100, 1,000 or their inverses; `drop_duplicates`; `impute(column)` with the reference median when nulls < 20%; `rename(mapping)`. Write the `repair_check` dry-run job.	Each repair fixes its fault in the dry run
W2 Thu	**LLM decision**: pass / warn / repair / quarantine.	Data faults decided correctly solo

When	Step	Done when
W2 Fri	**Answer `query.data_changed`:** compare the window's batches with the 5 before for that feature; if confirmed, post `incident.claim`, then `request.do_not_use` and `request.hold`.	Scenario C passes solo against the Monitor stub
W3	Fault runner (shared) + scenario runner; collect everyone's fault files. Expected change: new schema version for the changed feature, checked in a dry run, answer `done`.	All faults run from one command; scenario B step passes

9.5 What the LLM sees

Failed checks (counts, and statistics of failing rows, never the rows themselves), PSI table with importance ranks, Isolation Forest score, candidate repairs with their dry-run result, similar past incidents. Output: one menu action with the repair and its parameters.

9.6 Fake version, faults, tests, risks

- **Fake:** marks every pending batch `passed` and posts `data.batch_checked`; answers `query.data_changed` with a canned "unit change on total_sulfur_dioxide since batch X" (scenario C) or "no change".
- **Faults you add:** renamed column, missing values in alcohol, total sulfur dioxide in g/L (÷ 1,000), duplicated rows, a new column, pH values out of range, a batch of white wines (valid but different: should be a warning, not a quarantine).
- **Tests:** ≥ 90% of data faults detected with ≤ 1 false alarm per 15 normal batches; scenario C: claim, quarantine, `do_not_use` and `hold` within one batch cycle; repaired batches pass all checks.
- **Risks:** with only 100 rows a batch, PSI is noisy; use 10 bins and compare with several past batches, not just one. Your shared work (base loop) is needed on day 3 of week 1; the replay script can grow during week 2.

10. Agent 4: Trainer · owner P4

The Trainer retrains only when it is worth it, and forwards a new model only if it is better overall **and** not worse for any group of wines. Difficulty: medium. Runs on `request.retrain`, or after 10 new clean batches.

10.1 Scope

The baseline has no training code; the README trains the public MLflow example. In week 1 you add `train.py`: a scikit-learn `Pipeline(StandardScaler, ElasticNet)`, logged to MLflow and registered as one model `wine-quality`, plus the `reference/` files the Monitor and Data Checker need. Use the **same `train()` function in every system** we compare, so the experiment measures the agents, not a better algorithm.

Groups for the fairness check (no lookup files needed): `alcohol_band` (< 10, 10-11.5, > 11.5 % vol), `acidity_band` (fixed acidity < 7.5, 7.5-9.5, > 9.5), `quality_band` (true quality ≤ 5, 6, ≥ 7). A group counts only with at least 50 wines in the evaluation set.

The gate (thresholds in `agents.yaml`): overall MAE at least 2% better than the live model on the same evaluation set (the latest labelled clean batches, not used for training), **and** for every group the upper end of the bootstrap 95% interval of (new MAE − live MAE) below +5% of the live model's group MAE. Bootstrap: 500 resamples, a few lines of numpy.

MVP: retrain decision with a simple gain estimate, clean-window choice, training in a dry run, group gate, candidate registration with reference files, handoff to the Deployer, refusal and rejection messages. **Extras:** a learned gain predictor, small hyperparameter search.

10.2 Needs and gives

Needs	From	Stand-in
batch status, `do_not_use`, clean windows	Data Checker	sample registry
`request.retrain`	Monitor	Monitor stub
`request.expected_change` (retrain on post-change data)	Reviewer	Reviewer stub
delayed labels	label feeder	sample labels
release outcome (did the retrain pay off?)	Deployer	Deployer stub

Gives	To
MLflow versions of `wine-quality`, alias `candidate`	Deployer
`reference/`: `reference.parquet` (holdout rows with features, prediction, label, groups), `feature_importance.json`, `psi_bins.json`, `group_metrics.json`	Monitor, Data Checker
`request.deploy_candidate`	Deployer
`training.decision`, `model.candidate_rejected`	Monitor, human
`request.clean_window`	Data Checker
dry-run runner and LLM helper (shared)	everyone

10.3 Action menu

Action	Level	Success check
`refuse`	1	`training.decision` posted with expected gain and threshold
`request_clean_window`	1	Data Checker answers with batch ids
`retrain_in_dry_run`	2	run finished, metrics and comparison logged
`register_candidate`	1	new version with alias `candidate` and all reference files
`send_to_deployer`	1	Deployer answers `accepted`
`reject_candidate`	1	`model.candidate_rejected` posted with failing groups

10.4 Build steps

When	Step	Done when
W1 Mon-Tue	Write `train.py` and train once; register `wine-quality` version 1 with alias `champion`. Produce `samples/reference/`.	Version 1 in MLflow; API loads it by alias
W1 Wed-Fri	**Dry-run runner (shared):** subprocess, temp folder, `DRY_RUN=1`, timeout, MLflow experiment `dry_run`. **LLM helper (shared):** section 5.9. Agree the `request.deploy_candidate` payload with P5 and P6.	Every agent can call both
W2 Mon	`train(window) -> TrainResult`, pure: read clean batches, build features with `prepare_features_train`, fit, log to MLflow with the reference files, register a version.	Version appears with all files
W2 Tue	**Evaluation module:** overall and per-group MAE; bootstrap comparison of new vs live on the same holdout.	Comparison report for two versions
W2 Wed	**Triggers:** `request.retrain`, or ≥ 10 new clean batches since the last training. **Window:** all clean batches, or the last 10/20, excluding quarantined batches and, after an expected change, everything before it. **Gain estimate v0:** `gain = a × weighted_psi + b × mae_rise + c × log(new_rows)` with hand-set a, b, c; refuse below `min_gain_pct`.	Scenario E passes solo
W2 Thu-Fri	**LLM decision:** retrain or not, which window, forward or reject, using memory of past retrains. `train_compare` dry-run job; `register_candidate`; `request.deploy_candidate`.	Scenarios D, E, F pass solo; Deployer stub accepts

When	Step	Done when
W3	Integration with the Data Checker (clean window), the Monitor (requests) and the Deployer (handoff, outcome).	Scenario D end to end

10.5 What the LLM sees

Request reason, estimated gain with its inputs, threshold, available clean windows with row counts, last training time, any active expected change, similar past retrains and whether they paid off, and after training the comparison report (overall and failing groups only). Output: `refuse`, `retrain_in_dry_run` with the window, `send_to_deployer` or `reject_candidate`.

10.6 Fake version, faults, tests, risks

- **Fake:** answers `request.retrain` with a `training.decision` (retrain or refuse, by fixture) and posts `request.deploy_candidate` pointing at a hand-registered version.
- **Faults you add** (applied to the labels by the harness): gradual concept drift (wines above 11.5% alcohol scored 1 point lower, ramping in over 10 batches), sudden drift (all scores −1 from a batch on), group-specific damage (high-alcohol wines removed from a training window), bad candidate (trained on a corrupted window).
- **Tests:** E refused with a logged reason; F rejected although better on average; D retrained and forwarded; fewer useless retrains than the "retrain every 10 batches" baseline.
- **Risks:** concept drift changes the labels, not the inputs, so the Data Checker cannot see it; that is intended and is how D differs from C. With 1,600 rows, the evaluation set is small: keep the group minimum at 50 and report intervals, not single numbers.

11. Agent 5: Deployer · owner P5

The Deployer decides how much live traffic a new model gets, moves it forward step by step, and rolls back fast when it misbehaves. Difficulty: second hardest. Runs when a candidate arrives and during every rollout.

11.1 Scope: one app, two models

The baseline API serves one model, has no prediction log and gives every new model 100% of traffic at once. Instead of adding a router and a second container (heavy), we extend the **same FastAPI app**:

- At startup and whenever `release_state` changes (checked every 5 s), it loads `models:/wine-quality@champion` and, if set, `models:/wine-quality@candidate`. Both are small, so both fit in memory.
- For each request it picks the variant: with probability `candidate_share` the candidate, otherwise the champion. In **shadow** mode the champion answers and the candidate's prediction is only logged.
- Every prediction is written to `predictions` (batched every second) with `variant` and `model_version`.
- The existing Prometheus metrics get a `variant` label, so Grafana and the Deployer can compare latency and errors per model.
- An admin endpoint `POST /admin/release` (only the Deployer calls it) changes the share immediately, without waiting 5 s.

The traditional pipeline keeps the original behaviour: new model straight to 100%, no automatic rollback.

MVP: the extended app, prediction log, stage machine shadow → 10% → 50% → 100% (human), statistical comparison, automatic rollback on hard limits, human approval, answers to `query.deployment_changed`. **Extras:** traffic forecasting, scaling with replicas, pinned code + model release.

11.2 Stages and gates

Stage	Leave the stage when	Gate checked
Shadow	≥ 300 shadow predictions, ≥ 100 of them with labels	absolute errors not worse (one-sided Mann-Whitney U, p < 0.05, from scipy)
10%	≥ 30 simulated minutes and ≥ 150 candidate requests	p95 latency ≤ 1.2 × champion; error rate ≤ champion + 0.5 point; errors not worse
50%	same as 10%	same, plus a `traffic_replay` at 3× speed in a dry run

Stage	Leave the stage when	Gate checked
100%	a human approves `request.approval` with the comparison summary	human

Hard limits are code, not LLM. If the candidate's error rate > 5% or its p95 latency > 300 ms for two consecutive minutes, the Deployer sets its share to 0 at once and reports afterwards. This is safe without asking because it only returns to a model a human already approved.

11.3 Needs and gives

Needs	From	Stand-in
`request.deploy_candidate` + alias `candidate`	Trainer	a hand-registered version
`request.hold`	Monitor, Data Checker	stubs
`request.expected_change` (code + model together)	Reviewer	Reviewer stub
live requests and delayed labels	replay script	sample files
`human.decision` for 100%	P6 CLI	auto-approve in stub mode
dry-run job `traffic_replay`	P4 runner, own job	in-process

Gives	To
`predictions` table	Monitor
`release_state` table	the app, Monitor, Trainer
alias `champion` in MLflow	Monitor, Data Checker
`deployment.stage_changed`, `deployment.rolled_back`	Monitor, Trainer
answers to `query.deployment_changed`	Monitor
`request.approval` for 100%	human

Gives	To
experience log (shared)	everyone

11.4 Action menu

Action	Level	Success check
`hold`	1	share unchanged; reason posted
`rollback_emergency`	1 (hard limit only, plain code)	candidate share 0; error rate back to champion level within 2 minutes
`rollback`	2	same, decided by the LLM below the hard limit
`advance_stage` (up to 50%)	2	dry-run replay passed; no hard-limit breach for 10 minutes after
`promote_100`	3	human approved; alias `champion` moved; old champion kept for rollback

11.5 Build steps

When	Step	Done when
W1 Mon-Tue	Run the baseline API; measure `/predict` latency with `simulate_requests.sh`; note memory with `docker stats`. Switch model loading to the alias.	Numbers in your README
W1 Wed-Fri	**Experience log (shared):** SQLite table, `remember`, `similar`, `best_fixes`, in-memory fallback. Agree `predictions` and `release_state` with P4 and P6. Make `samples/predictions.parquet`.	Memory used by two other agents
W2 Mon	**Extend the app:** load champion + candidate, share and shadow logic, prediction log, `variant` label on metrics, admin endpoint.	Monitor reads real rows; share moves live

When	Step	Done when
W2 Tue	**Metrics reader:** PromQL over HTTP to Prometheus (p95 with `histogram_quantile`, error rate per variant) and SQL on `predictions` joined with `labels`.	Comparison computed for a test rollout
W2 Wed	**Stage machine**, gates and hard-limit rollback as plain code, with unit tests on made-up metrics.	Rollback fires in tests within 2 simulated minutes
W2 Thu-Fri	**LLM decision** for advance / hold / rollback in the grey zone. Human approval path for 100%. `traffic_replay` dry-run job.	Bad candidate held at 10% solo; promotion only after `human.decision`
W3	Integration: candidate from the Trainer, holds from the Monitor and Data Checker; scenarios D and G. Pinned code + model release for scenario B (if time; otherwise a human card).	D and G pass end to end

11.6 What the LLM sees

Stage and time in stage, candidate vs champion comparison (latency, errors, error distributions and the test result), sample sizes, holds requested by others, similar past rollouts. Output: `hold`, `advance_stage` or `rollback`; never `promote_100` without a human.

11.7 Fake version, faults, tests, risks

- **Fake:** answers `request.deploy_candidate` with `accepted`, posts a shadow → 10% → 50% sequence of `deployment.stage_changed`, answers `query.deployment_changed`.
- **Faults you add:** candidate latency spike (an env var `SLOW_MS=400` that sleeps in the candidate path), candidate errors on 10% of requests, traffic surge (× 5 for 10 minutes, from the harness).
- **Tests:** rollback within 2 minutes of a hard-limit breach; a bad candidate held at 10%; 100% only after human approval; zero unsafe actions.

- **Risks:** the LLM's role is thin here because canary analysis is a known pattern. Keep the LLM to the grey zone (neither clearly fine nor over the hard limit) and say so honestly in the report.

12. Agent 6: Monitor · owner P6

The Monitor tells whether the live model is getting worse, even before the true scores arrive, and finds out why by asking the agents upstream. Most scenarios start here. Difficulty: medium. Runs continuously.

12.1 Two signals: early and late

The baseline's Grafana shows curves but never explains them. The Monitor reads the prediction log and keeps two signals:

- **Early signal (no labels needed):** drift on the inputs, weighted by feature importance, plus an **estimated MAE**. The estimate is a small model trained once on the reference set to predict the *absolute error* from the 11 features and the prediction (a scikit-learn `GradientBoostingRegressor`, about 30 lines, trains in a second). On each window, the average predicted error is the estimated MAE.
- **Late signal:** the **real MAE**, computed when the true scores arrive (`labels.delay`, 1 simulated day by default).

The early signal only sees problems caused by changed inputs. It cannot see concept drift (same wines, different scores), which only the late signal catches. That difference is exactly what separates scenario C from scenario D.

MVP: windows over the prediction log, weighted drift, estimated MAE, real MAE, per-group checks, flag logic with deduplication, root-cause questions with deadlines, retrain and hold requests, expected-change handling. **Extras:** trend warning before the threshold, learning which features give false alarms, extra Grafana panels for the agents.

12.2 Needs and gives

Needs	From	Stand-in
`predictions`	Deployer	sample
`labels`	label feeder	sample

Needs	From	Stand-in
reference data, importance, PSI bins, group metrics	Trainer	`samples/reference/`
`release_state`, `deployment.*`	Deployer	Deployer stub
answers to the three `query.*`	Checker, Reviewer, Deployer	stubs
`data.batch_checked`, `data.drift_trend`	Data Checker	stub
`request.expected_change`	Reviewer	stub
`training.decision`	Trainer	stub
dependency map	P2	sample

Gives	To
`monitor.health_report`, `monitor.degradation_detected`	Checker, Trainer, Deployer
`request.retrain`	Trainer
`request.hold`	Deployer
`query.data_changed`, `query.code_changed`, `query.deployment_changed`	Checker, Reviewer, Deployer
board, message format, catalogue, results log, human cards and CLI (shared)	everyone

12.3 Action menu

Action	Level	Success check
`report`	1	finding posted; at most one human card per incident
`ask_upstream`	1	answers received, or timeouts noted in `evidence`
`request_retrain`	1	Trainer answers in time
`request_hold`	1	Deployer answers `accepted`

Action	Level	Success check
`mark_expected`	1	only with an active `request.expected_change`; alerts on that feature muted until the new model is live
`confirm_by_replay`	2	dry-run replay gives the real MAE on recent labelled data

12.4 Build steps

When	Step	Done when
W1 Mon-Tue	Draft the board tables and message model before the contract sprint.	Draft ready on Wednesday
W1 Wed-Fri	**Board (shared):** SQLAlchemy tables on `board.db`, `post`, `read`, `ask`, `answer`, `thread`, `heartbeat`, subscription matching, in-memory board, a `board tail` command to watch messages live. Message model and catalogue checks. Results tables. Human cards and CLI.	**M1 Hello board**
W2 Mon	**Windows:** predictions joined with labels available at `clock.now()`, per simulated hour, per group.	Window table for one simulated day
W2 Tue	**Drift:** PSI per feature against the reference bins; weighted drift = Σ importance × PSI / Σ importance. **Estimated MAE:** fit the error model on the reference set; estimate per window; compute the real MAE when labels arrive; log the gap between the two (a result in itself).	Estimated vs real MAE plot
W2 Wed	**Flag logic:** estimated MAE > reference × 1.10 for 3 windows, or real MAE > reference × 1.10, or weighted drift > 0.2, overall or per group. Cooldown and one incident per problem through `incidents.open_or_join`.	⩽ 1 flag per fault, none on normal days

When	Step	Done when
W2 Thu-Fri	**Root-cause flow:** walk the dependency map upstream from the top drifting features; send the three `query.*` questions at once with deadlines; follow an `incident.claim` if one arrives; otherwise the LLM chooses among report, hold, retrain, confirm by replay. `confirm_replay` dry-run job.	Scenarios C and D pass solo
W3	Expected change: mark drift on the named feature as expected; after the new model goes live, load its reference. Integration: C and D first (they carry the most messages), then A-H.	Scenario B passes; M3

12.5 What the LLM sees

Estimated and real MAE vs reference, top drifting features with importance, affected groups, answers (or timeouts) to the three questions, active expected changes, live version and rollout stage, similar past incidents and whether they were real. Output: one menu action. Rule: `request_retrain` only when the Data Checker reports clean data; otherwise the data must be fixed first.

12.6 Fake version, faults, tests, risks

- **Fake:** posts `monitor.degradation_detected` from fixtures (variants for C and D), sends `request.retrain`, `request.hold` and the three questions. The Checker, Trainer and Deployer owners rely on it most, so ship it early in week 1.
- **Faults you add:** the API returns a constant prediction (silent failure); a change in label delay; the prediction log loses a column.
- **Tests:** C: asks the Data Checker, follows its claim, sends no retrain request; D: detects the drop through real labels and requests a retrain; detects faster than the fixed-threshold baseline; at most one human card per incident.
- **Risks:** alert storms (one incident per branch of the dependency map and time window, plus a cooldown per feature). The board is on everyone's critical path: ship the in-memory board on day 1 of the contract sprint.

13. Integration, testing and experiments

Tests come in five layers. The first three run on every merge; pairs and full scenarios run in week 3.

Layer	What it proves	Where	Who writes it
Unit	your statistics and actions are right on made-up inputs	`agents/<name>/tests/`	agent owner
Contract	every message you post and every example you read matches the catalogue	`make contract-test`	sender writes examples, receiver reviews
Solo scenario	your agent does the right thing in A-H with the other five as fakes	`make solo AGENT=… SCENARIO=…`	agent owner
Pair	two real agents agree on fields, timing and tables	week 3 sessions	both owners
Full scenario	all six real agents reach the expected end state	`make scenario S=…`	team

13.1 Definition of done for each agent (end of week 2)

1. Runs with `make solo AGENT=<name>` against the fakes of the other five, and inside `docker-compose.agents.yml` from the shared image.
2. Reads settings only from `agents.yaml` and environment variables; no hard-coded hosts, paths or thresholds.
3. Posts only its catalogue kinds; at least one example per kind in `contracts/examples/`.
4. Handles every kind it subscribes to; ignores unknown kinds with a log line; never crashes on a bad message; handlers are idempotent.
5. Uses the shared clock, board, LLM helper, dry run, memory and dependency map, with no private copies.
6. Every LLM call, action, check and incident outcome is logged; `rule_decision()` works with the LLM switched off.

7. Its fake and fixtures cover all its outgoing kinds and canned answers.
8. Its solo scenarios pass; README lists run commands, thresholds and known limits.

13.2 Integration week (week 3)

Integrate along the busiest message paths first, two agents at a time, with the others still as fakes. Each session ends with its scenario passing.

Day	Session	Scenario that must pass
Mon AM	Data Checker + Monitor (questions, claims, holds)	C
Mon PM	Trainer + Deployer (candidate handoff, stages, rollback)	F, G
Tue AM	Monitor + Trainer + Data Checker (retrain loop, clean windows)	D up to the candidate, E
Tue PM	Reviewer + Setup Agent (CI, rebuild) and Reviewer + Monitor (code questions)	A
Wed	Reviewer + the four others for the migration plan	B, H
Thu	All six real agents, fresh `board.db`	A, B, C, D (**M3**)
Fri	Fixes; E-H with all six; feature freeze	E, F, G, H (**M4**)

13.3 Common integration failures and how the rules prevent them

Symptom	Usual cause	Prevention
an agent misses messages after a restart	read position kept in memory	`cursors` table; idempotent handlers
three agents handle the same problem; alert storm	each opens its own incident	`incidents.open_or_join()`; cooldowns; one claim per incident
an agent waits forever	unanswered question	`expires_at` on every question; `ask()` returns `None`
events in the wrong order	local, real and replay time mixed	UTC everywhere; only `clock.now()`

Symptom	Usual cause	Prevention
receiver crashes on a payload	sender changed a field	catalogue checks on post and read; contract tests in CI
a dry run overwrites real data	same table or model alias	`DRY_RUN=1` guard in `tables.py`; MLflow experiment `dry_run`
loop stops on a bad LLM answer	invalid JSON or unknown action	check, one retry, rule fallback
SQLite "database is locked"	six agents writing at once	WAL mode on, short transactions, retry on lock

13.4 Fault file format

Every fault is a YAML file plus two functions, `inject` and `revert`. The runner writes the ground truth to the `faults` table, which agents never read.

```
# harness/faults/data/unit_change_total_so2.yaml
id: data.unit_change.total_sulfur_dioxide
layer: data                  # code | data | model | serving | monitoring
author: P3
target: {column: total_sulfur_dioxide, factor: 0.001}   # mg/dm3 -> g/dm3
start: {after_batches: 10}
duration: {batches: 3}
applies_to: [batch, requests]
inject: harness.faults.data:rescale_column
revert: harness.faults.data:restore_column
expected:
  owner: checker
  root_cause_node: "feature:total_sulfur_dioxide"
  acceptable_actions: [quarantine, repair]
  must_not: [request.retrain, promote_100]
```

13.5 Changing the contract after week 1

- Open a merge request labelled `contract` that changes `shared/` and `catalogue.yaml` together, with updated examples.
- Approval from P6 (catalogue keeper) and from the owner of every agent that sends or receives the affected kind.

- Adding an optional field or a new kind: version 1.1, merge any day. Renaming, removing or adding a required field: version 2.0, agreed at the Friday check.
- Never change a payload "just for now" inside your own agent; the contract tests will catch it anyway.

13.6 Experiments (week 4)

The same replay, fault library and metrics run every system; only the config changes.

System	Config	How it is built	Required?
1. Traditional pipeline	`system: traditional`	all agents with `reason: rules`, board off: fixed thresholds, retrain every 10 batches, new model straight to 100%, no automatic rollback	yes
2. Six agents	`system: agents`	this design	yes
3. Single LLM agent	`system: single_agent`	one agent whose menu is the union of all six and whose context is all six analyses	optional

Each run gets a fresh `board.db` and a `run_id`. Metrics are computed afterwards by joining the `faults` table with `messages`, `actions` and `llm_calls`. Repeat each fault **3 times** per system, because LLM decisions vary between runs. One run replays 40 batches of 100 wines, drawn with replacement from the 1,600 red wines (the first 10 batches are always clean, so baselines settle). At 12 simulated minutes per real second, one run takes a few minutes, so the whole experiment fits in one day.

Metrics to report:

Metric	Meaning
Detection rate	share of injected faults that some agent flagged
Time to detect	simulated minutes from injection to the first flag
Correct owner	share of incidents claimed by the expected agent
Time to recover	from injection to `incident.closed` with a good outcome

Metric	Meaning
Wasted retrains	retrains whose candidate was rejected or gave no gain
Unsafe actions	actions listed under `must_not`
Human cards per incident	how often a person was bothered
LLM cost	calls, tokens and latency per incident; share of `llm_invalid` fallbacks

14. Scenarios A-H

Each scenario is a file in `scenarios/` naming a fault, the expected owner, the messages that must appear (in any order unless stated), messages that must not appear, and a deadline in simulated time. The same file runs solo (other agents as fakes) and in full integration.

	Situation (wine version)	Owner	Must appear	Must not appear
A	**Accidental unit change in code:** the serving side divides total sulfur dioxide by 1,000 (mg → g), training does not	Code Reviewer	`code.change_assessed` with verdict ask_author or block, ratio ≈ 0.001 reported	`request.expected_change`; any retrain
B	**Intended unit change:** both sides, schema and commit tag changed to g/dm³	Code Reviewer	`request.expected_change`; four `done` answers; `request.deploy_candidate`; `request.approval`	Monitor degradation alert on that feature
C	**Unit change in the data feed:** incoming wines have total sulfur dioxide in g/dm³	Data Checker	`incident.claim` by the Checker; `request.do_not_use`; `request.hold`; one human card	`request.retrain`; more than one card

	Situation (wine version)	Owner	Must appear	Must not appear
D	**Real concept drift:** tasters score high-alcohol wines 1 point lower (labels change, inputs do not)	Monitor → Trainer	`monitor.degradation_detected` (source: labels); `query.data_changed` answered "no"; `request.retrain`; `request.deploy_candidate`; stages up to 50%; `request.approval`	quarantine of clean batches
E	**Drift on a low-importance feature** (e.g. residual sugar × 1.5)	Trainer	`training.decision` = refuse, expected gain below threshold	a training run beyond the estimate
F	**Candidate better on average, worse for high-alcohol wines**	Trainer	`model.candidate_rejected` listing `alcohol_band: >11.5`	`request.deploy_candidate`
G	**Candidate latency spike** (`SLOW_MS=400`) or error burst during the 10% stage	Deployer	`deployment.rolled_back` (automatic) within 2 minutes	`promote_100`
H	**Fix of the baseline's `predict()` bug:** named input fields + a clean 422 error instead of the NameError	Code Reviewer	`code.change_assessed` with approve_with_migration; `request.expected_change` (replay script, Monitor parser, Deployer); pinned release of the new API	`block`

14.1 Scenario file example

```
# scenarios/C.yaml
id: C
title: Unit change in the data feed
fault: data.unit_change.total_sulfur_dioxide
deadline: {simulated_hours: 6}
expect:
```

```
  owner: checker
  messages:                     # any order
    - {kind: incident.claim, sender: checker}
    - {kind: request.do_not_use, sender: checker, to: [trainer]}
    - {kind: request.hold, sender: checker, to: [deployer]}
  either:                       # who notices first depends on timing; both
are fine
    - [{kind: query.data_changed, sender: monitor}]
    - [{kind: data.batch_checked, sender: checker, payload: {status:
quarantined}}]
  must_not:
    - {kind: request.retrain}
  max_human_cards: 1
end_state:
  batches: {"batch-0011": quarantined}
  release_state: {stage: unchanged}
```

Why timing-tolerant expectations matter. In scenario C the Monitor may see live errors before the Data Checker checks the batch, or the other way round. Both are correct. Write expectations as "these messages, in any order" plus "one of these paths", or scenarios will fail for the wrong reason during integration.